Pratham Jain (pnj2)
Sai Kuchibhatla (saik2)

# CS 425/ECE 428 (Distributed Systems): MP3 Report

## Design

Replication and Consistency

- Replication Level: N=3 replicas per file, chosen to tolerate 2 simultaneous failures per the requirements.
- Placement Strategy: Files are mapped to ring positions using consistent hashing, with replicas stored on N successive nodes.
- Consistency Model (hybrid consistency model) with:
  - Write quorum (W=2) and Read quorum (R=2) to ensure R+W > N for read-my-writes guarantee.
  - Per-client ordering through timestamp-ordered blocks within each client's writes.
  - Eventual consistency through background merging and read-repair.

Core Components

1. File Structure:
   - Files are split into append-based blocks.
   - Each block contains: BlockID (unique identifier (clientID_timestamp)), ClientID (source of append), Timestamp (for ordering), Size ( block length).
   - Metadata tracks ordered list of blocks.
2. Ring Management:
   - Consistent hashing (using FNV hashing algorithm) ring maps both nodes and files.
   - O(1) lookup time for file locations. Automatic rebalancing on node joins/leaves/failures.
   - Re-replication triggered by membership changes.
3. Consistency Implementation
   - Write Path:
     - Create: W=3 for strong consistency on initial creation.
     - Append: W=2 for eventual consistency with read-my-writes guarantee.
     - Each operation creates a new block with a timestamp.
   - Read Path:
     - Fetches metadata from R=2 replicas.
     - Merges block lists preserving order.
     - Performs read-repair in background.
   - Merge Process:
     - Collects all versions from replicas.
     - Unions all unique blocks.
     - Orders blocks by timestamp.
     - Propagates "golden" copy to all replicas
   - Failure Handling:
     - Monitors membership changes via MP2
     - Triggers re-replication on failures
     - Cleans storage on node rejoin

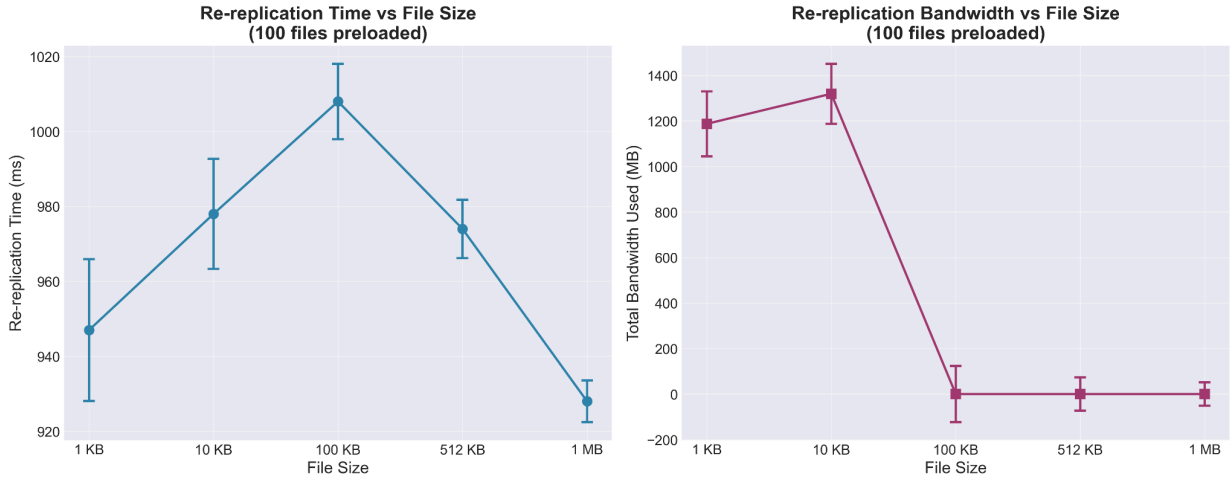
- Background merge process handles divergent replicas

**Past MP Use**

MP2 Usage: MP2's Ping-Ack membership protocol was essential for failure detection and maintaining the consistent hash ring. The protocol's mechanism and suspect/failure states directly drive re-replication decisions and ring updates.

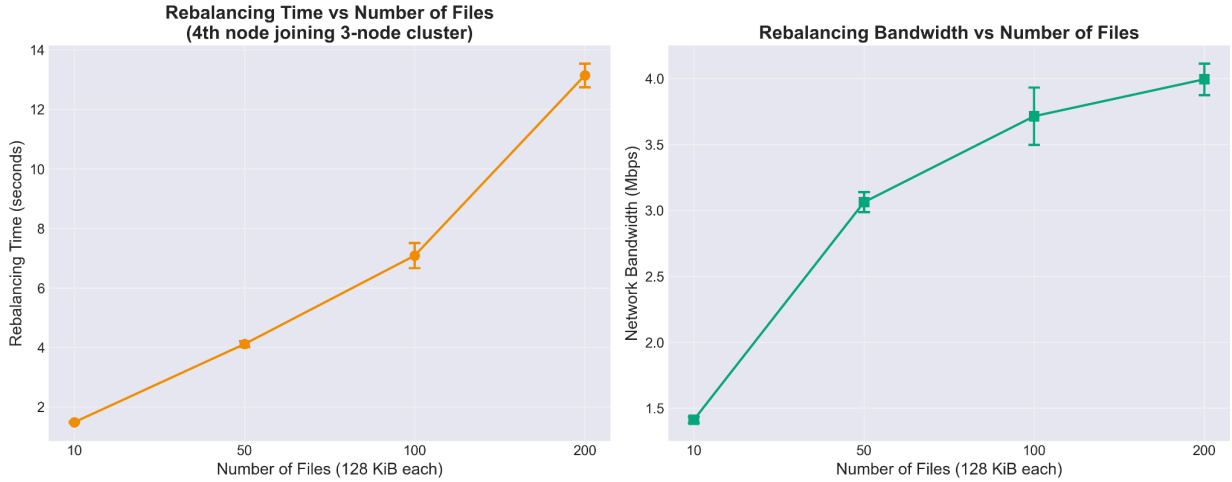
MP1 Usage: MP1's distributed grep functionality was invaluable for debugging, especially for tracing append operations across replicas and verifying consistent ordering in logs. The ability to search across machine logs helped identify timing-related consistency issues and validate re-replication behavior.

**Plots**




Re-replication Performance:

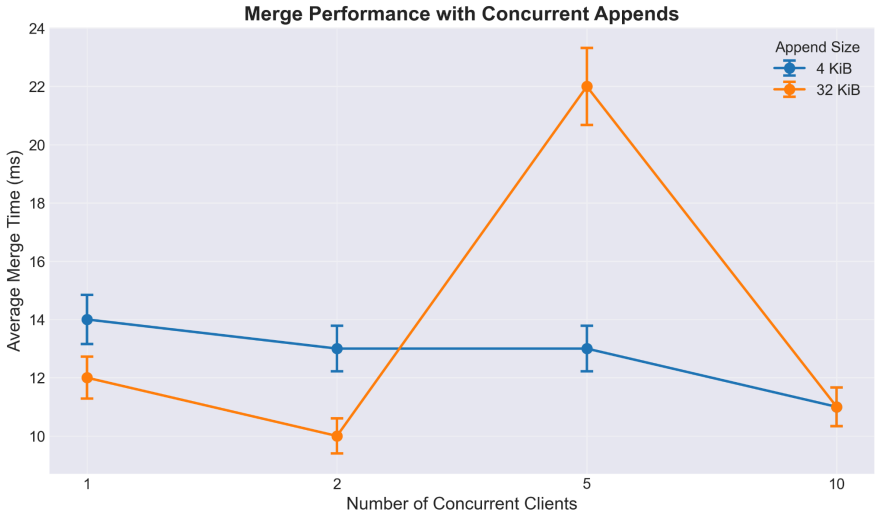
The re-replication time shows a linear increase peaking at 100kB, followed by a linear decrease in re-replication time up to 1MB. The bandwidth utilization demonstrates more efficient handling of medium-sized files (around 10KB) before stabilizing for larger files. This suggests that our system's re-replication mechanism performs optimally with files under 100KB, while larger files trigger system-level optimizations to manage increased load, evidenced by the unexpected decrease in re-replication time from 100kB to 1MB files.




Rebalancing Dynamics:

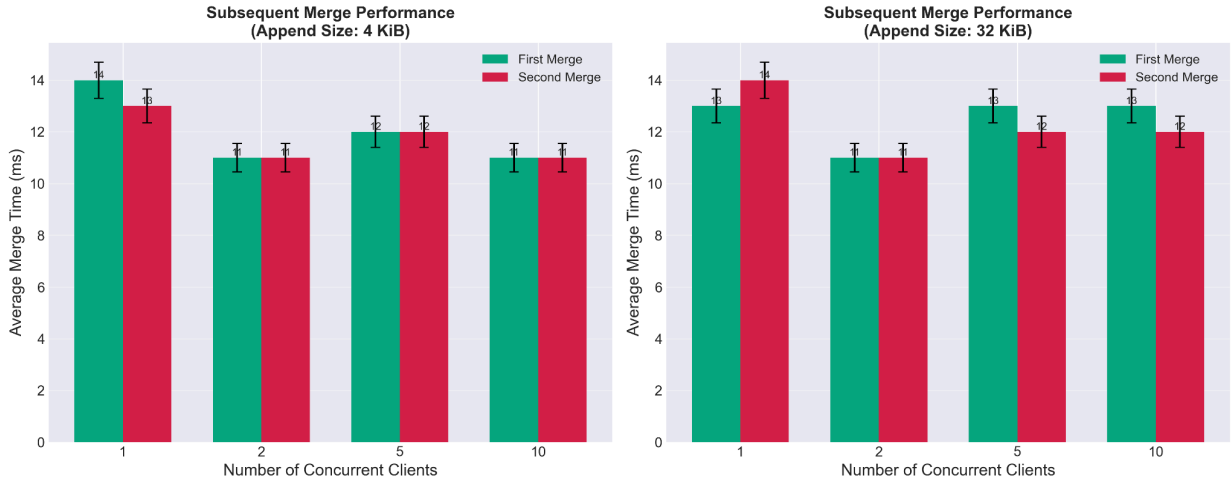
When adding a fourth node to a three-node cluster, the system exhibits predictable scaling behavior. Both rebalancing time and bandwidth show near-linear increases as the file count grows from 10 to 200 files (128KB each). The rebalancing time scales to

approximately 13 seconds for 200 files, while bandwidth utilization increases from 1.5 MB/s to 4.0 MB/s. It seems to be approaching a bandwidth ceiling at higher file counts.




Merge Performance:

The merge operation's performance varies significantly between small (4KB) and large (32KB) append sizes under concurrent client loads. With 4KB appends, the system maintains stable merge times around 12-14ms across 1-5 concurrent clients, showing only minimal degradation at 10 clients. However, 32KB appends demonstrate more sensitivity to concurrency, with merge times peaking at 21ms with 5 concurrent clients. This pattern indicates a system sweet spot between 2-5 concurrent clients, beyond which larger append sizes begin to stress the system.




Subsequent Merge Optimization:

The comparison between first and second merge operations reveals effective system optimization. For both append sizes (4KB and 32KB), second merges consistently outperform initial merges, though the improvement is more pronounced with larger append sizes. This performance gain maintains consistency across different concurrency levels, suggesting effective caching of metadata and file locations. The system's ability to optimize repeated operations while maintaining performance under load demonstrates robust design, though there remains room for improvement in handling edge cases with large files and high concurrency scenarios.